

Aperture: Zero-Selector Fine-Grained Reactivity for React via Proxy-Based Read Tracking

Aparajith N.

Independent Research

aparajith.n@gmail.com

Abstract

React state libraries that share data across components face a re-render granularity problem: naive shared state (useState lifted into a parent, or an undifferentiated Context) re-renders every subscriber on every update, while the standard fix, a selector function per call site as in Redux and Zustand, requires the dependency to be declared manually, and a forgotten selector fails silently, falling back to over-rendering with no error. We present Aperture, a dependency-free React store (aperture-store) that infers each component’s data dependency automatically instead of requiring it to be declared. Aperture hands each render a Proxy standing in for the store’s state, records which keys are actually read off that Proxy during that render, and uses the recorded read set, not a hand-written selector, to decide, on the next store update, whether the component’s snapshot changed, via useSyncExternalStore for concurrent-safe integration with React’s own render lifecycle. This tracking idea is not new: react-tracked [4] pioneered automatic state-usage tracking for React in 2020, as a layer over Context and Redux using deep object tracking. Aperture’s contribution is a from-scratch, minimal reimplement of the same idea as a standalone, dependency-free, flat-key store, built to understand the mechanism rather than to supersede it. We evaluate the design on a controlled benchmark comparing three re-render strategies (naive re-render-all, a hand-selected store, and Aperture) across three workload scales. Aperture reproduces the hand-selected store’s re-render count exactly (verified by direct equality assertion) in every scenario tested, with zero selectors authored at any call site, while the naive baseline over-renders by up to 83.5x at the largest tested scale (200 components, 100 fields, 500 updates: 100,200 vs. 1,200 renders). The complete implementation, including the Proxy-tracking hook and its useSyncExternalStore integration, compiles to 421 bytes minified and brotli-compressed, with no runtime dependency beyond React itself.

Keywords: React, state management, fine-grained reactivity, JavaScript Proxy, dependency tracking, useSyncExternalStore, selector-free stores

1 Introduction

React does not ship an opinion about state shared across components that are not in a direct parent-child relationship. The ecosystem’s answer has converged on an external store accessed through a hook, but implementations split on one design question: how does a subscriber communicate which slice of shared state it depends on, so an update to an unrelated slice does not force it to re-render?

Two answers dominate current practice. The first is to not distinguish at all: a single useState lifted into a common ancestor, or one undifferentiated React Context, re-renders every consumer on every change to the shared value, regardless of which field an individual consumer actually reads. The second is to require an explicit selector at each call site: useSelector in Redux, useStore(s => s.field) in Zustand, which fixes the over-rendering problem by diffing the selector’s result, not the whole state object, between renders. This is correct, but the dependency is a manual annotation: omitting or mis-writing a selector does not throw or warn, it silently falls back to the first behavior.

Aperture explores a third point in this design space: infer the dependency from what a component actually reads during the render in which it read it, and require no selector to be written at all. The read set is captured as a side effect of a Proxy get trap firing during ordinary property destructuring, the same work a component already performs to know what to display, and useSyncExternalStore [1] is used to fold the resulting re-render decision into React’s own concurrent-safe render lifecycle, rather than a hand-rolled subscription that can tear under concurrent rendering. This inference mechanism itself is prior art, not a novel discovery: react-tracked [4] has shipped Proxy-based state-usage tracking for React since 2020 (§2.2). What this paper contributes is not the idea but a minimal, from-scratch, standalone execution of it, evaluated on its own terms.

This paper makes the following contributions:

1. A minimal, dependency-free reimplement of Proxy-based automatic read-tracking (the mechanism react-tracked [4] introduced for React in 2020), as a standalone flat-key store requiring no selector function or dependency array at any call site.

2. An integration of that mechanism with `useSyncExternalStore`, giving the resulting re-render decision the same concurrent-rendering correctness guarantees React’s own state primitives have, and, as a direct consequence, working server-side rendering and hydration support with no additional code.
3. A controlled benchmark (`bench/re-render-bench.test.tsx`) isolating the re-render-count effect of automatic tracking versus hand-written selectors versus no tracking at all, with a direct equality assertion between the automatic and hand-selected strategies rather than an informal comparison.
4. A complete open-source implementation (61 lines, `src/createStore.ts`) shipping at 421 bytes minified and brotli-compressed, with zero runtime dependencies beyond a React peer dependency, available at the project’s repository under the MIT License.

2 Related Work

2.1 Selector-Based Stores

Redux, via React-Redux’s `useSelector` [2], and Zustand [3] both solve the re-render granularity problem by requiring a selector function at each subscribing call site; the store diffs the selector’s return value across renders (by reference or a supplied equality function) rather than the whole state object. This is the mechanism Aperture’s benchmark selector strategy (§6) reimplements in miniature specifically to isolate “selector present vs. absent” as the single variable under study. Redux additionally provides a middleware system, time-travel debugging via DevTools, and a normalized-store convention for large applications; Zustand omits the Provider/reducer boilerplate Redux requires but keeps the same per-call-site selector requirement. Neither infers the dependency from the read itself.

2.2 Proxy-Based Reactivity

`react-tracked` [4] is the closest prior work to Aperture, and the one this report’s framing must be read against: since 2020 it has provided automatic Proxy-based state-usage tracking for React, using the proxy-compare library to record which parts of a state object a component reads during render and bail out re-renders for components that did not read the changed part, the same core inference Aperture performs. `react-tracked` is built as a layer over an existing state mechanism (`useReducer/Context`, or a `Redux/Zustand` store it wraps) and tracks state usage deeply, including nested objects. Aperture is not a new solution to the problem `react-tracked` already solved; it is a from-scratch, standalone reimplement of the same idea, deliberately narrower in two ways: it is not a layer over another state mechanism but a self-contained pub-sub store with

no dependency beyond React, and it tracks only the top-level keys of a flat state object rather than deep-/nested paths, avoiding the extra tracking library and the edge cases deep tracking introduces (circular references, proxy identity across nested reads). The value of building it was in understanding the mechanism end to end, not in improving on `react-tracked`’s design.

Valtio [5] is also built on a JavaScript Proxy [9] intercepting property reads, but applies the idea to state mutation rather than read tracking alone: its proxies wrap the entire state tree and permit direct in-place mutation, with reactivity following from the same read-interception mechanism `react-tracked` and Aperture use for the re-render decision.

2.3 Atom-Based Reactivity

Jotai [6] and Recoil [7] take dependency tracking to the level of individually declared atoms rather than keys within one shared object: each atom is its own subscription unit, so a component reading one atom is structurally isolated from updates to a different atom, without requiring a selector over a larger object. This achieves fine granularity through decomposition of the state itself rather than through inference over an existing object’s shape, and requires state to be authored as a set of atoms rather than as one object, which is a different modeling commitment than the single flat object Aperture and the selector-based stores in §2.1 both assume.

2.4 Framework-Level Reactivity

`Solid.js` [8] makes fine-grained reactivity a first-class primitive of the rendering model itself: any signal read inside a tracking scope is automatically subscribed, and updates re-run only the specific computation that read the changed signal, with no virtual DOM diff involved at all. This is the same “track what is read, not what is declared” idea Aperture applies, but implemented at the framework level with compiler and runtime support, rather than as a library layered on top of a rendering model (React’s) that does not provide automatic dependency tracking natively. Aperture’s contribution is showing the same idea is obtainable as a library-level pattern within React’s existing primitives (Proxy, `useSyncExternalStore`), without a compiler step.

3 Background

3.1 The Re-Render Decision in React

A React component subscribed to an external store re-renders when React’s reconciler determines the value returned by the store’s snapshot function differs, by reference, from the value returned on the previous render. Concretely, `useSyncExternalStore`(`subscribe`, `get-`

Snapshot, getServerSnapshot) [1] calls getSnapshot() on every render and after every subscribe notification, and schedules a re-render only if the returned reference changed. The entire re-render-granularity problem in §1 reduces to one question: what should getSnapshot return, and when should that reference change?

- Naive (re-render-all): getSnapshot returns the entire state object by reference; any setState produces a new object, so every subscriber’s snapshot reference changes on every update, regardless of which field changed.
- Selector-based: getSnapshot returns selector(state); the reference changes only when the selector’s result changes across an update, which requires the developer to have written a selector that reads exactly the fields the component displays.
- Aperture: getSnapshot returns a previously-issued Proxy reference unchanged if every key actually read off it since the last render is still Object.is-equal to the live state, and a fresh Proxy otherwise, without any selector function existing at the call site.

3.2 The Tracked Read Set

For a given render of a component, the store keeps a small map: for every key the component actually reads off the Proxy during that render, it records the key and the value it read. That map, the tracked read set, is built incrementally, purely as a side effect of the Proxy’s get trap firing on each property access; nothing about it is declared in advance the way a selector would be. On the next call to getSnapshot, the store walks that same map against the live (possibly updated) state and checks, key by key, whether the current value still matches what was recorded. If every tracked key still matches, nothing the component read has changed, so it does not re-render. If even one tracked key’s value has changed, the component re-renders. That check is exactly the guarantee a correctly hand-written selector over the same fields would provide; §6 tests that equivalence empirically rather than just asserting it, since the description above says nothing about implementation cost or edge cases (§7.2).

4 Aperture System Design

Aperture’s runtime consists of a plain pub-sub store (getState, setState, subscribe, holding the store’s state) and the useStore() hook, in which the tracking mechanism of §3.2 is implemented. The complete hook is 33 lines; the mechanism decomposes into two operations, given as Algorithms 1 and 2 below and implemented directly as the get trap and getSnapshot in src/createStore.ts.

Algorithm 1 TrackedRead (Proxy get trap)

Require: property key accessed on the render Proxy, live state, per-render tracked map (initially empty)
 Ensure: the property’s current value, with the access recorded

```

1: value ← state[key]
2: trackedMap[key] ← value
3: return value

```

Algorithm 2 getSnapshot (re-render decision)

Require: previously tracked map from the last render (or empty, on first render), live state
 Ensure: a Proxy reference: reused if unchanged, freshly minted otherwise

```

1: if tracked map is empty then
2:   return MakeProxy(state)  ▷ primes the tracked
   map, returns new Proxy
3: end if
4: for each key, lastValue in tracked map do
5:   if state[key] has changed from lastValue then
6:     return MakeProxy(state)  ▷ tracked key
   changed: re-render
7:   end if
8: end for
9: return previousProxy ▷ all tracked keys unchanged:
   no re-render

```

MakeProxy(state) allocates a fresh Map for the new render’s tracked set and a fresh Proxy whose get trap is Algorithm 1 closed over that Map, matching Reflect semantics via Reflect.get(state, prop, receiver) so the Proxy is behaviorally transparent for anything beyond property reads. “Changed” in Algorithm 2 is checked with Object.is, the same identity comparison React itself uses to compare hook values, so a value replaced with an equal primitive or the same object reference does not count as a change.

4.1 Integration with useSyncExternalStore

An earlier implementation stage used a hand-rolled useState/useEffect subscription in place of useSyncExternalStore. It was replaced specifically because useSyncExternalStore is the API React 18 provides for synchronizing a render with state that lives outside React in a way that remains correct under concurrent rendering: a hand-wired subscription can tear, meaning different components in the same commit observe different versions of the external state, which useSyncExternalStore’s contract rules out. getSnapshot (Algorithm 2) also serves directly as the server-snapshot argument, which is what makes renderToString and hydrateRoot work with no additional code: the read-tracking mechanism has no browser-API dependency, so it produces an identical tracked set and snapshot whether it runs during a server render or a client one.

4.2 Implementation History

The design in §4 was arrived at over four deliberate stages, each motivated by a concrete failure mode of the prior stage rather than by starting from the finished mechanism: (1) a naive store that re-renders every subscriber on every `setState`, establishing the baseline cost being solved; (2) manual per-subscriber selectors, fixing over-rendering but introducing the forgotten-selector failure mode of §1; (3) Proxy-based automatic tracking (§3–4) replacing the manual selectors; (4) the hand-rolled subscription swapped for `useSyncExternalStore` (§4.1), closing the concurrent-rendering tearing gap stage 3 alone does not address.

5 Implementation

Aperture is implemented in TypeScript with React 18’s `useSyncExternalStore` as its only runtime integration point, and no runtime dependency beyond a `react >=16.8.0` peer dependency. The public surface is a single factory, `createStore(initialState)`, returning the `useStore` hook augmented with `getState`, `setState`, and `subscribe`. The store holds a flat state object; `setState` accepts either a partial object or an updater function and merges the result with a shallow spread, matching the immutable-update convention of the selector-based stores in §2.1. The package builds to both ESM and CJS outputs via `tsup`, ships hand-written `.d.ts` type declarations, and is marked `sideEffects: false`. No build-time code generation or compiler transform is involved; the entire mechanism is expressible in standard ES2015+ (Proxy, Reflect) plus one React 18 hook.

6 Experiments

6.1 Experimental Setup

Environment. All experiments run under Vitest 2.1 with the `jsdom` environment and React Testing Library 16, on the author’s development machine, invoked via `npm run test` and `npx vitest run bench/rerender-bench.test.tsx`. No real browser DOM or GPU is involved: this is a CPU-bound, single-process JavaScript benchmark, not a systems benchmark, and §7.2 discusses what that scope does and does not license.

Workload. We construct a synthetic access pattern designed to isolate the re-render decision described in §3.1 from confounds like multi-key reads or conditional rendering: a fixed number of components, each reading exactly one of a fixed number of state fields (assigned round-robin), subjected to a sequence of updates that each set one field, cycling through all fields in order. Three scenarios vary the field count, component count, and update count together to test whether the qualitative findings hold as scale grows: 10 fields / 10 compo-

nents / 200 updates, 50 / 50 / 200, and 100 fields / 200 components / 500 updates.

Backends compared. (i) naive: a single `useState` lifted above a `Context.Provider`; every consumer re-renders on every update regardless of which field it reads. (ii) selector: a minimal hand-rolled `useSyncExternalStore` + selector store, structurally the mechanism §2.1’s libraries are built on; this is the correctness ceiling, since a component re-renders if and only if the field its selector reads changed. (iii) aperture: `createStore` from `src/createStore.ts` (§4–5), with no selector authored at any call site. All three backends are exercised against the identical workload in the same process, back to back, per scenario.

Metrics. Total renders: summed render count across all components after the full update sequence, counted by direct instrumentation in each component’s render body (not sampled or inferred). Elapsed time: wall-clock milliseconds for the `act()`-wrapped update loop, measured with `performance.now()`. Bundle size: `dist/index.js`, minified and brotli-compressed, measured with `size-limit`.

6.2 Re-Render Count

Table 1 reports total re-render count and elapsed time for all three backends across all three scenarios (source: `bench/rerender-bench.test.tsx`).

Table 1: Total re-renders and elapsed time by backend and scenario.

Fields	Components	Updates	Backend	Total Renders	Elapsed (ms)
10	10	200	naive	2,010	16.1
10	10	200	selector	210	4.2
10	10	200	aperture	210	4.0
50	50	200	naive	10,050	20.8
50	50	200	selector	250	4.4
50	50	200	aperture	250	4.2
100	200	500	naive	100,200	138.4
100	200	500	selector	1,200	25.7
100	200	500	aperture	1,200	23.5

aperture’s total render count is identical to selector’s in all three scenarios; the benchmark test asserts this equality directly (`expect(aperture.totalRenders).toBe(selector.totalRenders)`) rather than reporting it as an incidental observation, so the claim in §1 that automatic tracking reproduces hand-selected re-render behavior exactly is a passing assertion in the repository, not a description of a single favorable run.

6.3 Over-Rendering Growth

Figure 1 shows total renders per backend across the three scenarios on a log scale.

total renders (log scale)	
naive [K=10,N=10]	##### 2,010
selector [K=10,N=10]	## 210
aperture [K=10,N=10]	## 210
naive [K=50,N=50]	##### 10,050
selector [K=50,N=50]	## 250
aperture [K=50,N=50]	## 250
naive [K=100,N=200]	##### 100,200
selector [K=100,N=200]	## 1,200
aperture [K=100,N=200]	## 1,200

Figure 1: Total renders per backend across scenarios (log-scaled bar widths). The naive backend’s bar grows with both the component count and the update count, since every component re-renders on every update; selector and aperture grow only with the sum of the two, and are visually indistinguishable from each other at every scale tested.

The over-rendering ratio (naive divided by tracked) grows from 9.6x at the smallest scenario to 40.2x at the middle scenario to 83.5x at the largest (100 fields, 200 components, 500 updates): 100,200 renders for naive against 1,200 for both tracked backends. This growth is expected from the workload definition (§6.1): naive’s render count is component count multiplied by update count, regardless of which field changed, while the tracked backends’ render count is just component count plus update count (one mount render per component, one additional render per update for the single affected component). Multiplying two numbers that both scale up grows much faster than adding them, so the gap between naive and the tracked backends widens without bound as the scenario grows.

6.4 Timing

Wall-clock elapsed time (Table 1) tracks render count roughly linearly across backends, as expected. aperture and selector are statistically indistinguishable in elapsed time at every scale tested here: the largest gap observed is 2.2 ms, within run-to-run noise for a jsdom-hosted process, indicating that the Proxy get trap and the per-render tracked-key walk in Algorithm 2 do not register as a measurable cost relative to the selector store’s equivalent work, at the scales this benchmark covers. §7.2 discusses the limits of what this single-machine, single-run timing measurement licenses concluding.

6.5 Bundle Size

Table 2: Compressed bundle size, dist/index.js (minified, brotli).

Metric	Value
Configured size-limit budget	1,024 B
Measured size	421 B
Headroom under budget	58.9%

The entire implementation (store, Proxy-tracking hook, and useSyncExternalStore integration) compiles to 421 bytes compressed, with zero runtime dependencies beyond React itself. No comparison figures for other libraries’ bundle sizes are reported here as measured data: this report only asserts numbers produced against this repository’s own build, not figures recalled for other packages, which would be unverified in this context.

6.6 Correctness Test Suite

Independent of the benchmark, sixteen pre-existing tests (test/createStore.test.ts, test/rerender.test.tsx, test/ssr.test.tsx, test/hydration.test.tsx) assert re-render behavior directly against real React components via React Testing Library, counted rather than eyeballed from a demo: a component reading an unrelated key never re-renders; a component reading multiple keys re-renders on a change to any of them; setting a tracked key to its current value does not re-render (exercising the Object.is comparison in Algorithm 2, not just key presence); React 18 StrictMode’s synthetic double-invoke mount/unmount/remount cycle changes neither the outcome nor leaves a dangling subscription; and renderToString/hydrateRoot produce output with no hydration-mismatch warning, confirming §4.1’s claim that the mechanism has no hidden browser-API dependency.

```
(check) test/createStore.test.ts (8 tests)
(check) test/rerender.test.tsx (5 tests)
(check) test/ssr.test.tsx (2 tests)
(check) test/hydration.test.tsx (1 test)
(check) bench/rerender-bench.test.tsx (1 test)

Test Files 5 passed (5)
Tests 17 passed (17)
```

7 Discussion

7.1 When Automatic Tracking Matches a Hand-Selected Store

§6.2’s result, exact render-count parity with selector in every scenario verified by assertion, follows from Algorithm 2 implementing the same decision rule as a correctly-written selector (§3.1) over the same key set,

by construction. The practical value is not that the decision rule differs, but that writing it correctly requires no selector at all: the read set (§3.2) is populated by ordinary destructuring the component already performs to render, so there is no call site at which the dependency can be forgotten or mis-declared, the failure mode identified for selector-based stores in §1 and §2.1.

7.2 Threats to Validity

The benchmark in §6 is a single representative run rather than an average over multiple trials with variance reported, on one machine, under jsdom rather than a real browser DOM: absolute timings (§6.4) should not be read as production numbers, and are reported only to support the relative claims (render-count parity; the over-rendering growth in §6.3) that the equality assertion in the benchmark test verifies directly regardless of timing noise. The workload (single-field readers under uniformly distributed single-field updates) is a controlled, synthetic access pattern chosen specifically to isolate the re-render decision from confounds; real component trees mix multi-key reads, conditional reads, and reads inside callbacks (§7.3), none of which this benchmark models. No comparison was run against the actual Zustand or Redux packages: the selector backend is a minimal reimplement of the same underlying mechanism, chosen so the comparison isolates “selector present vs. absent” from implementation differences unrelated to that question, at the cost of not benchmarking those libraries’ actual shipped code.

7.3 Known Limitations

Reads outside a render body are not tracked. The Proxy’s get trap (Algorithm 1) only fires on an actual property access, and Algorithm 2 only has a tracked set to compare because it was built during a render. A read inside a `useEffect` or an event handler still returns the live, current value (the Proxy always reads straight from current state), but cannot be the reason the component re-renders, since nothing the component rendered depended on it. This is a scope boundary, not staleness, with a direct existing answer: `useStore.getState()` for a current value that should not trigger a re-render, or reading the value in the render body (even if only used inside an effect) when a re-render on change is wanted.

State is flat by design. Nested-object mutation in place is not detected; updates must spread a new nested object in, the same requirement any immutable-update store has. Deep tracking (comparable to Valtio, §2.2) is explicitly out of scope, not a planned extension.

No middleware, devtools, or cross-tab sync. Aperture targets a handful of components sharing state without hand-written selectors, not large-application state management with time-travel debugging or async action orchestration: a stated scope boundary rather than an

incompleteness.

8 Conclusion

Automatic Proxy-based read tracking, combined with `useSyncExternalStore` for the re-render decision itself, reproduces the exact re-render behavior of a correctly hand-selected store on the workloads tested in §6, with no selector authored at any call site, in 421 bytes of compressed code with no runtime dependency beyond React. The over-rendering a naive shared-state implementation exhibits under the same workload grows without bound as component count and update count scale together (§6.3), reaching 83.5x at the largest scale tested here, which is the concrete cost automatic or manual dependency tracking is solving. The mechanism’s scope boundary, reads outside a render body are not tracked (§7.3), is a direct consequence of what a Proxy’s get trap can observe, with an existing, direct answer rather than a workaround. The benchmark in §6 is deliberately narrow, isolating the one question this report set out to answer; broader claims (real applications, a real browser DOM, comparison against the actual shipped Zustand/Redux packages rather than a minimal reimplement of their mechanism) are left to further work, as scoped in §7.2.

Acknowledgements

This work received no external funding.

References

- [1] React Team. `useSyncExternalStore`. React documentation, 2022.
- [2] D. Abramov et al. Redux: A predictable state container for JavaScript apps. redux.js.org, 2015.
- [3] D. Kato et al. Zustand: A small, fast, and scalable state-management solution. github.com/pmndrs/zustand, 2019.
- [4] D. Kato. react-tracked: State usage tracking for React. github.com/dai-shi/react-tracked, 2020.
- [5] D. Kato et al. Valtio: Proxy-state made simple. github.com/pmndrs/valtio, 2021.
- [6] D. Kato et al. Jotai: Primitive and flexible state management for React. github.com/pmndrs/jotai, 2020.
- [7] Facebook Open Source. Recoil: A state management library for React. recoiljs.org, 2020.

- [8] R. Carniato et al. SolidJS: Simple and performant reactivity for building user interfaces. solidjs.com, 2018.
- [9] ECMA International. ECMA-262: ECMAScript Language Specification, Proxy Objects. Section 10.5, current edition.

A Data Availability

The Aperture source code, the correctness test suite (test/), and the benchmark harness used to produce Table 1 and Figure 1 (bench/re-render-bench.test.tsx) are available in this repository under the MIT License. Appendix E gives the exact commands used to regenerate every number in §6.

B Author Contributions

A.N.: Conceptualization, Methodology, Software, Formal Analysis, Writing (Original Draft), Writing (Review & Editing).

C Conflict of Interest

The author declares no conflicts of interest.

D Ethics Declaration

This research involves no human subjects, personal data, or sensitive data. No ethics approval was required.

E Reproducing the Results in This Report

```
npm install
npm run test
# 17 tests: 16 correctness (Sec. 6.6) + 1 benchmark (Sec. 6.2)
npx vitest run bench/re-render-bench.test.tsx
# benchmark table only (Table 1, Sec. 6.2)
npm run size
# bundle size (Table 2, Sec. 6.5)
```